

# **OPTIMISATION OF THEME PARK NAVIGATION VIA PATH-FINDING**

**21CSC203P/ADVANCED PROGRAMMING PRACTICE  
PROJECT REPORT**

*Submitted by*

**KAVYA BALAKRISHNAN RA2411003020454**

**PRAGYA REDDY A RA2411003020473**

**KRITHIV SHANKAR RA2411003020489**

**Under the guidance of**

**MR. C Santhosh Kumar**

**(Assistant Professor, Department of Computer  
Science and Engineering)**

*in partial fulfilment for the award of the degree*

*of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**

*Of*

**FACULTY OF ENGINEERING AND TECHNOLOGY**

**SCHOOL OF COMPUTER SCIENCE ENGINEERING**



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY  
RAMAPURAM, CHENNAI - 600089**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**  
**(Deemed to be University U/S 3 of UGC Act, 1956)**

**BONAFIDE CERTIFICATE**

Certified that this project report titled **OPTIMISATION OF THEME PARK NAVIGATION VIA PATH-FINDING** is the bonafide work of **KAVYA BALAKRISHNAN (RA2411003020454), PRAGYA REDDY A (RA2411003020473), KRITHIV SHANKAR (RA2411003020489)** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

**Mr. C Santhosh Kumar M.E.,(Ph.D.),**  
**Assistant Professor**  
Computer Science and Engineering,  
SRM Institute of Science and Technology,  
Ramapuram, Chennai.

SIGNATURE

**Dr. J. Sutha M.E.,(Ph.D.),**  
**Professor and Head**  
Computer Science and Engineering,  
SRM Institute of Science and Technology,  
Ramapuram, Chennai.

Submitted for the project viva-voce held on \_\_\_\_\_ at SRM Institute of Science and Technology, Ramapuram, Chennai- 600089.

# **SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**

**RAMAPURAM, CHENNAI - 89**

## **DECLARATION**

We hereby declare that the entire work contained in this project report titled **OPTIMISATION OF THEME PARK NAVIGATION VIA PATH-FINDING** has been carried out by **KAVYA BALAKRISHNAN (RA2411003020454), PRAGYA REDDY A (RA2411003020473), KRITHIV SHANKAR (RA2411003020489)** at SRM Institute of Science and Technology, Ramapuram, Chennai, under the guidance of **Mr. C Santhosh Kumar, Assistant professor**, Department of Computer Science and Engineering.

**Place: Chennai**

**Date:**

**KAVYA BALAKRISHNAN**

**PRAGYA REDDY A**

**KRITHIV SHANKAR**

# ABSTRACT

Maze solving is a classic problem that demonstrates the application of logical paradigms to real-world challenges. Using the logical programming paradigm, maze solving can be represented as a problem where the maze structure, paths, and constraints are encoded as logical facts and rules. Logical inference mechanisms are then used to explore multiple possible paths, backtrack when dead ends are reached, and ultimately derive a solution that connects the start to the destination. This paradigm allows the programmer to specify what solves a problem rather than how to compute it. An approach such as this, simplifies the modelling of complex search problems and also provides a foundation for developing systems capable of automated reasoning and decision-making.

# TABLE OF CONTENTS

|                                           | <b>Page .no</b> |
|-------------------------------------------|-----------------|
| <b>ABSTRACT</b>                           | 4               |
| <b>LIST OF FIGURES</b>                    | 6               |
| <b>LIST OF ACRONYMS AND ABBREVIATIONS</b> | 7               |
| <b>1 Introduction</b>                     | 8               |
| 1.1 Introduction                          | 8               |
| 1.2 Problem statement                     | 9               |
| 1.3 Objectives                            | 9               |
| 1.4 Scope and Motivation                  | 10              |
| <b>2 Existing System</b>                  | 12              |
| <b>3 Design and Proposed Methodology</b>  | 13              |
| <b>4 Implementation</b>                   | 17              |
| <b>5 Result</b>                           | 23              |
| <b>6 Conclusion</b>                       | 25              |
| References                                | 26              |

## **LIST OF FIGURES**

|                   |    |
|-------------------|----|
| 3.1 Block Diagram | 13 |
| 5.1 Fig 5.1       | 23 |
| 5.2 Fig 5.2       | 23 |
| 5.3 Fig 5.3       | 24 |
| 5.4 Fig 5.4       | 24 |

## **LIST OF ACRONYMS AND ABBREVIATIONS**

- Eg - Example
- OS – Operating System
- GUI - Graphical User Interface
- UAC – User Account Control
- JAR – Java Archive
- IDE – Integrated Development Environment
- JDK - Java Development Kit

# CHAPTER 1

## INTRODUCTION

### 1.1 INTRODUCTION

This report details the development of an optimized navigation solution designed for dynamic path planning within a complex environment, specifically a theme park setting. The project began by implementing and testing a fundamental graph search algorithm (such as Dijkstra's or A\*) to solve simple maze problems. This foundational solver was then scaled and adapted to address the intricacies of a real-world environment, resulting in the **Theme Park Path Planner (TPPP)**. The TPPP is an interactive tool that leverages graph representation, where attractions, rides, and key locations are represented as nodes, and the walking paths connecting them are represented as weighted edges. The TPPP employs a two-tiered optimization approach, using **Dijkstra's Algorithm** for segment-by-segment pathfinding and a **Greedy Nearest Neighbour Heuristic** to sequence the user-selected stops. The primary function of the TPPP is to minimize the total estimated time for users who have selected a specific subset of park attractions, culminating in the calculation of the shortest return journey from the entry point, through all chosen stops, and back to the exit.



## 1.2 PROBLEM STATEMENT:

The core problem addressed is the optimization of the visitor experience in a large-scale theme park through efficient routing. Given a set of user-selected attractions and a defined walking speed, the challenge is to determine the most time-efficient, non-redundant path. This path must start at the entry point, include every selected attraction, and return to the exit point (co-located with the entry point for this test case), while simultaneously minimizing total travel time and accounting for attraction durations and fixed operating hours. The system must overcome the computational complexity inherent in pathfinding across numerous nodes by using an effective heuristic to sequence the required stops.

## 1.3 OBJECTIVES:

1. **Develop a Robust Core Algorithm:** Successfully scale a foundational maze-solving algorithm into a generalized pathfinding engine capable of handling complex graph structures representative of a theme park layout.
2. **Optimize Path Sequence:** Calculate the most efficient path sequence for a route that sequentially visits a specified, non-ordered list of mandatory nodes (attractions) using an efficient optimization heuristic.
3. **Provide User Customization:** Incorporate user-defined variables, such as attraction selection and walking pace, to personalize the resulting route and estimated duration.

4. **Integrate Data Constraints:** Successfully integrate various park data points—including attraction descriptions, fixed durations, and associated fares—into the path calculation and presentation layer.
5. **Deliver a Complete Circuit:** Guarantee that the final route successfully begins at the park entry and concludes at the park exit, forming a complete circuit that covers all selected attractions.

## 1.4 SCOPE AND MOTIVATION:

### Scope

The project focuses exclusively on **geographical path optimization** within the defined theme park graph structure. The scope includes:

- Modelling the park environment as a weighted, undirected graph.
- Implementing a **Greedy Nearest Neighbour Heuristic** to solve the Traveling Salesperson Problem (TSP) variant, using Dijkstra's Algorithm for inter-node segment calculations.
- Presenting detailed attraction data (description, duration, fare) alongside the calculated route.
- The project specifically **excludes** dynamic, real-time factors such as current ride queue wait times, pedestrian congestion modelling, or weather conditions.

## **Motivation**

The primary motivation for developing the TPPP is to significantly enhance the visitor experience and demonstrate the real-world utility of graph theory. Visitors often waste time and energy navigating theme parks inefficiently, leading to fatigue and missed attractions. A tool that provides an optimized, personalized route minimizes travel distance, saves time, and maximizes the number of attractions a user can realistically visit in a day. Furthermore, this project serves as a compelling technical demonstration of how foundational computer science principles and heuristic optimization techniques can be applied to solve complex logistical challenges in the leisure and tourism industry.

## CHAPTER 2

### EXISTING SYSTEM

The most analogous commercial solutions to the TPPP are official itinerary and navigation platforms deployed by major theme park operators (e.g., Disney's Genie or Universal's mobile applications). These systems operate on the same fundamental principle: mapping the park as a weighted graph and utilizing efficient search algorithms (A\*, Dijkstra's) for inter-node pathfinding. They also incorporate a sequencing layer that addresses the mandatory-stop routing problem. However, a significant operational distinction exists in the data employed. While the TPPP is an academic, localized application dependent on static data (Java implementation reading local .txt files for fixed durations), commercial systems integrate **dynamic, real-time queue wait times** via networked APIs and server-side processing. Consequently, the optimization objective of commercial systems is to minimize **total time spent waiting in line**, whereas the TPPP's objective is a deterministic minimization of **geographical travel time** and **fixed activity duration**. In the immediate context of a localized, self-contained application, the TPPP serves as a direct computational replacement for the sub-optimal, manual itinerary planning process that visitors would otherwise employ using only a physical park map and publicly available attraction details.

# CHAPTER 3

## DESIGN AND PROPOSED METHODOLOGY

### 3.1 BLOCK DIAGRAM

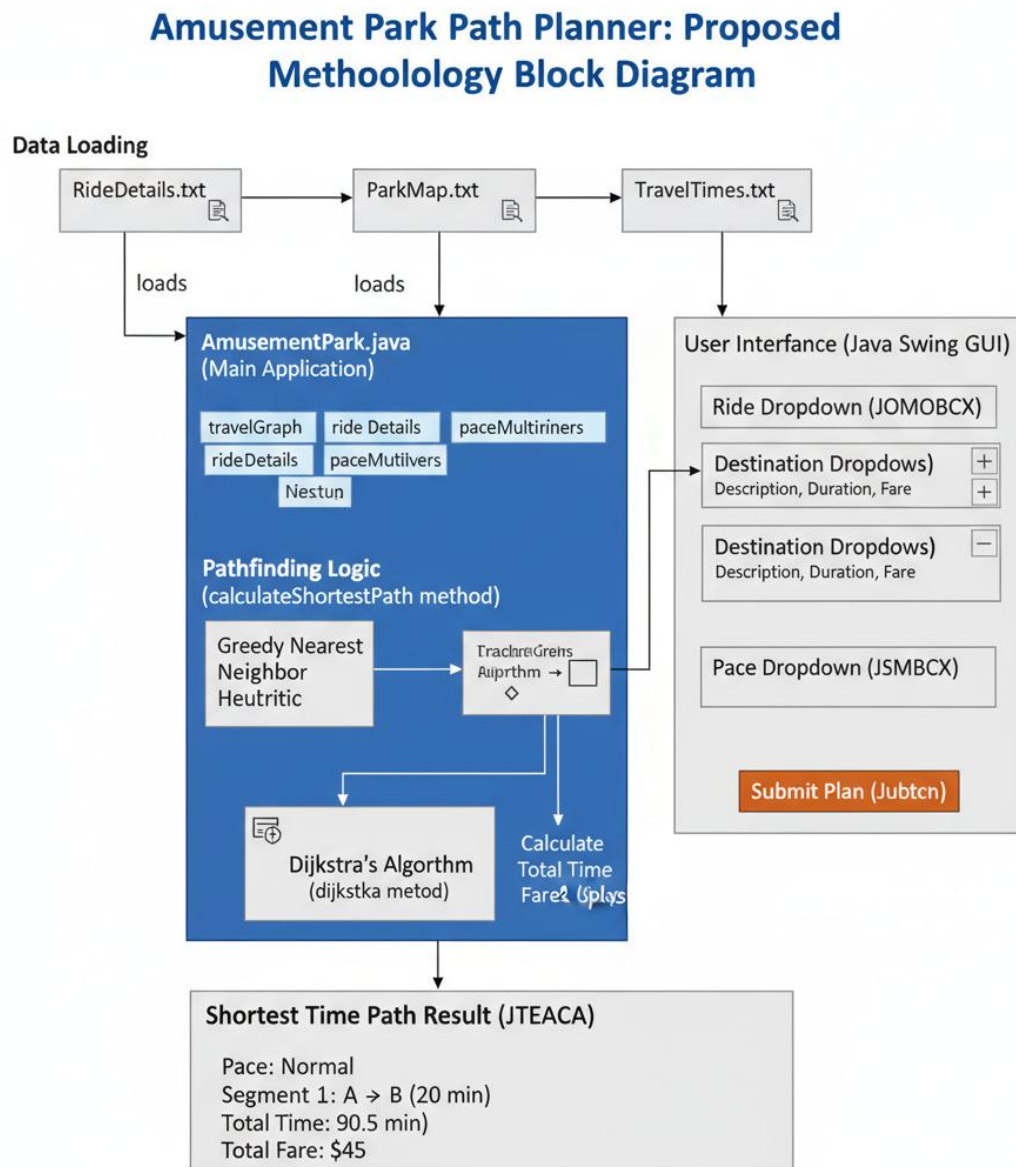


Fig 3.1 Block Diagram

## PROPOSED METHODOLOGY

The methodology for the Amusement Park Path Planner is structured around a sequential data processing and interaction flow, designed to deliver an optimized route based on user input.

1. **Data Ingestion:** Upon application initialization (AmusementPark constructor), the system first loads all static park data from designated text files. This includes:
  - RideDetails.txt: Populates a map (rideDetails) with ride names and their corresponding duration, fare, and description.
  - ParkMap.txt: Identifies all unique location nodes in the park.
  - TravelTimes.txt: Establishes the travelGraph (adjacency map) with base walking times between connected nodes and populates paceMultipliers for different walking speeds.
2. **User Interface (GUI) Presentation:** A Java Swing GUI is constructed, allowing the user to:
  - Browse ride details via a dropdown (rideDropdown).
  - Select multiple desired destination nodes (attractions) using dynamically added dropdowns (destinationDropdowns).
  - Choose a preferred walking pace (paceDropdown).
  - Visualize the park map (if park\_map.jpg is present), with zoom and pan functionality for context.

3. **Path Calculation Trigger:** The user initiates the path planning process by clicking the "Submit Plan" button. This action triggers the `calculateShortestPath()` method.
4. **Optimized Pathfinding (Two-Tiered Approach):**
  - **Greedy Nearest Neighbor Heuristic:** The `calculateShortestPath()` method employs a Greedy Nearest Neighbor heuristic. Starting from the `START_NODE` (Entrance), it iteratively determines the next unvisited destination by computing the shortest travel time to all remaining selected destinations. This involves repeated calls to Dijkstra's Algorithm.
  - **Dijkstra's Algorithm:** For each segment (from the current location to a candidate next destination), the `dijkstra()` method is invoked. This algorithm calculates the shortest path and its corresponding time, adjusted by the user-selected `paceMultipliers`, within the `travelGraph`.
5. **Total Time and Fare Accumulation:** As the greedy heuristic sequences the path:
  - The walking time from each `dijkstra()` segment is added to the `totalTime`.
  - The fixed duration of each visited attraction (plus a standard 5-minute wait) is also added to `totalTime`.
  - The fare of each visited attraction is added to `totalFare`.

- A final Dijkstra's calculation is performed to return from the last visited attraction back to the START\_NODE.
6. **Results Display:** The detailed path, including segments, total time, total fare, and the optimized sequence of visits, is then formatted and displayed in the pathResultArea on the GUI, providing the user with their personalized itinerary.



## CHAPTER 4

### IMPLEMENTATION

#### IMPLEMENTATION IN JAVA

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.event.MouseWheelEvent;
import java.io.*;
import java.util.*;
import java.util.List;
import javax.imageio.ImageIO;

// Class representing a node in the priority queue for Dijkstra's algorithm
class ParkNode implements Comparable<ParkNode> {
    public String name;
    public double time;
    public ParkNode(String name, double time) {
        this.name = name;
        this.time = time;
    }
    @Override
    public int compareTo(ParkNode other) {
        return Double.compare(this.time, other.time);
    }
}

// --- Data Loading Methods (Reads new node names from files) ---
private void loadData() {
```

```

try {
    loadRideDetails("RideDetails.txt");
    loadGraphAndTimes("ParkMap.txt", "TravelTimes.txt");
} catch (IOException e) {
    JOptionPane.showMessageDialog(this, "Error loading data files: " + e.getMessage(),
                                   "File Error", JOptionPane.ERROR_MESSAGE);
    e.printStackTrace();
    System.exit(1);
}
}

```

```

private void loadRideDetails(String filename) throws IOException {
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] parts = line.split("\\|");
            if (parts.length >= 4) {
                // Storing: [Duration, Fare, Description]
                rideDetails.put(parts[0].trim(), new String[]{parts[1], parts[2], parts[3]});
            }
        }
    }
}

```

```

private void loadGraphAndTimes(String mapFile, String timeFile) throws IOException {
    // Load all unique nodes from the map file
    Set<String> nodes = new HashSet<>();
    try (BufferedReader br = new BufferedReader(new FileReader(mapFile))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] parts = line.split("\\|");
            if (parts.length >= 2) {

```

```

        nodes.add(parts[0].trim());
        nodes.add(parts[1].trim());
    }
}
}
allDestinations.addAll(nodes);
Collections.sort(allDestinations);

// Load travel times and pace multipliers
try (BufferedReader br = new BufferedReader(new FileReader(timeFile))) {
    String line;
    while ((line = br.readLine()) != null) {
        String[] parts = line.split("\\|");
        String nodeA = parts[0].trim();
        String nodeB = parts[1].trim();

        if (nodeA.startsWith("PaceMultipliers")) {
            paceMultipliers.put(nodeB, Double.parseDouble(parts[2].trim()));
        } else if (parts.length >= 3) {
            double time = Double.parseDouble(parts[2].trim());
            // Add connection (undirected graph)
            travelGraph.computeIfAbsent(nodeA, k -> new HashMap<>()).put(nodeB,
time);
            travelGraph.computeIfAbsent(nodeB, k -> new HashMap<>()).put(nodeA,
time);
        }
    }
}

mapPanel.addMouseListener(new MouseAdapter() {
    @Override

```

```

public void mouseWheelMoved(MouseWheelEvent e) {
    double zoomFactor = 1.1;
    double newScale = scale;

    if (e.getWheelRotation() < 0) {
        newScale *= zoomFactor;
    } else {
        newScale /= zoomFactor;
    }

    newScale = Math.max(1.0, newScale);
    newScale = Math.min(5.0, newScale);

    if (newScale != scale) {
        scale = newScale;
        mapPanel.repaint();
    }
}
});

```

```

private DijkstraResult dijkstra(String start, String end, double multiplier) {
    Map<String, Double> minTime = new HashMap<>();
    Map<String, String> predecessor = new HashMap<>();
    PriorityQueue<ParkNode> pq = new PriorityQueue<>();

    minTime.put(start, 0.0);
    pq.add(new ParkNode(start, 0.0));

    while (!pq.isEmpty()) {
        ParkNode current = pq.poll();
    }
}

```

```

String u = current.name;

if (current.time > minTime.getDefault(u, Double.MAX_VALUE)) {
    continue;
}

if (u.equals(end)) {
    break;
}

Map<String, Double> neighbors = travelGraph.get(u);
if (neighbors != null) {
    for (Map.Entry<String, Double> neighbor : neighbors.entrySet()) {
        String v = neighbor.getKey();
        double weight = neighbor.getValue() * multiplier;
        double newTime = minTime.get(u) + weight;

        if (newTime < minTime.getDefault(v, Double.MAX_VALUE)) {
            minTime.put(v, newTime);
            predecessor.put(v, u);
            pq.add(new ParkNode(v, newTime));
        }
    }
}

List<String> path = new LinkedList<>();
String step = end;
while (step != null) {
    path.add(0, step);
    if (step.equals(start)) break;
    step = predecessor.get(step);
}

```

```

    }

    double finalTime = minTime.getDefault(end, Double.MAX_VALUE);

    return new DijkstraResult(finalTime, path);
}

private static class DijkstraResult {
    double time;
    List<String> path;
    public DijkstraResult(double time, List<String> path) {
        this.time = time;
        this.path = path;
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new AmusementPark());
}
}

```

# CHAPTER 5

## RESULT

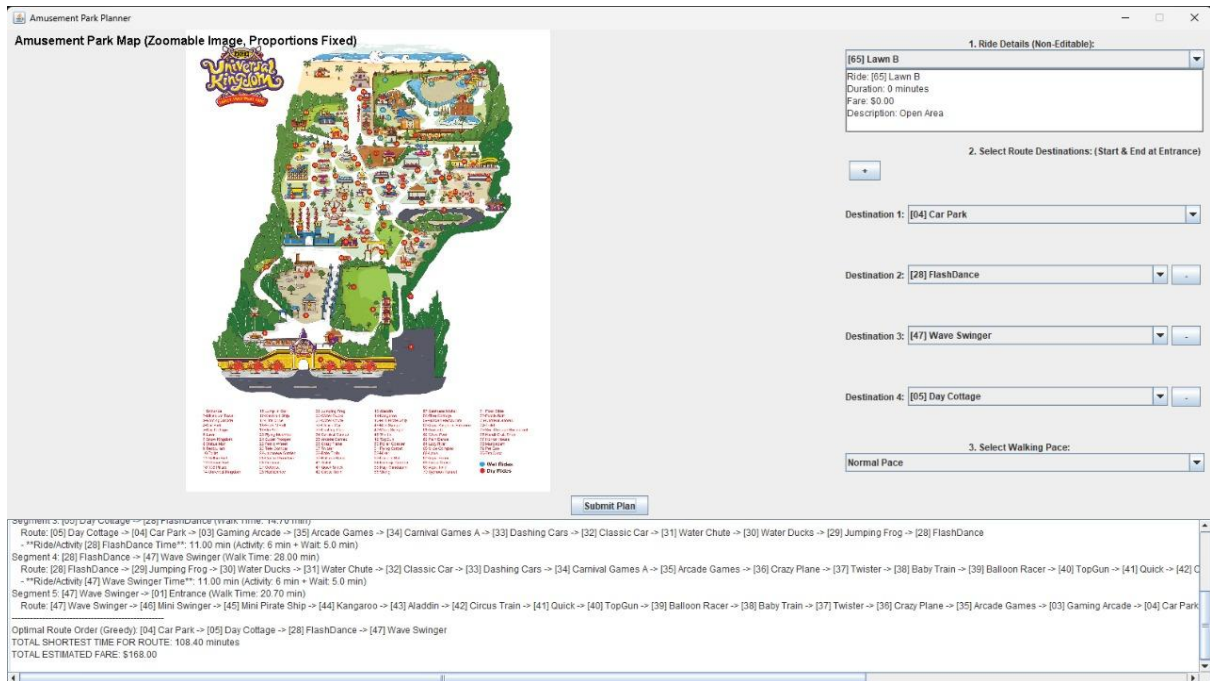


Fig 5.1

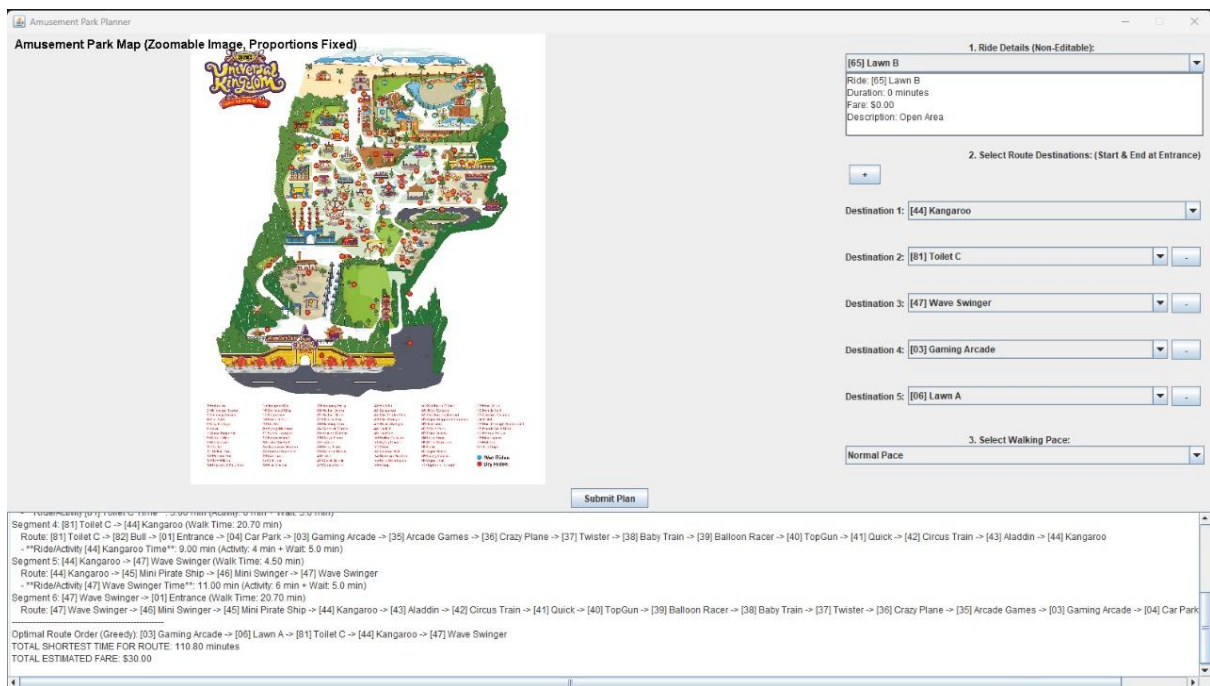


Fig 5.2

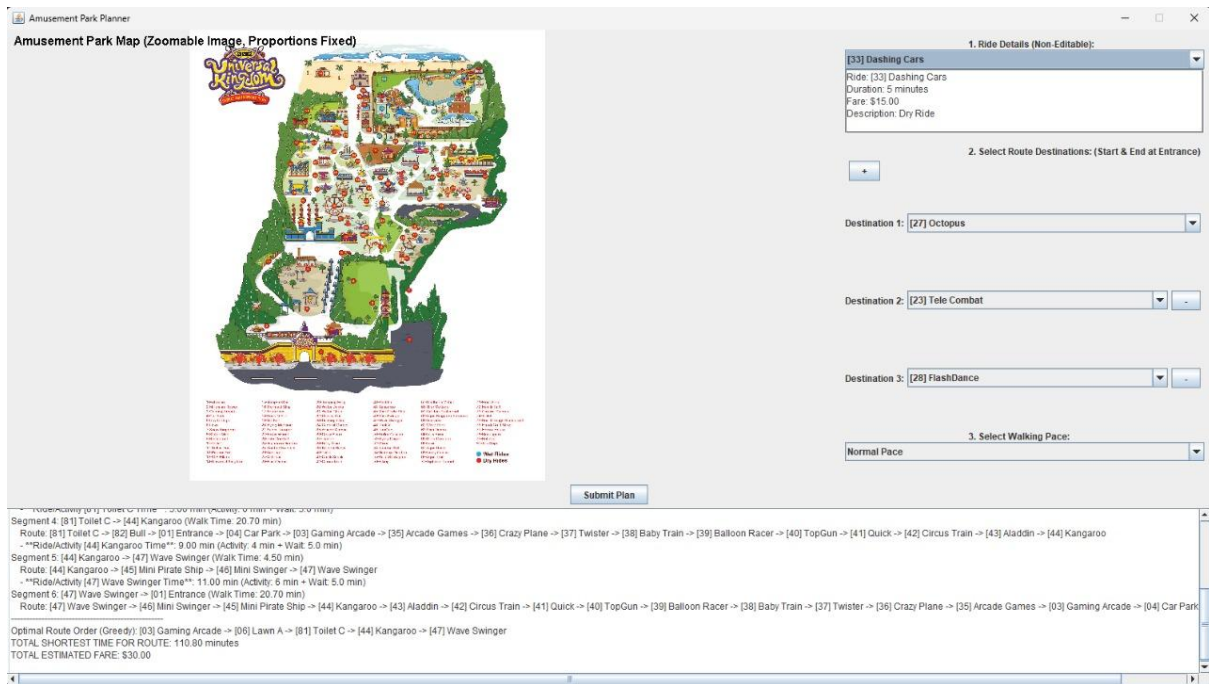


Fig 5.3

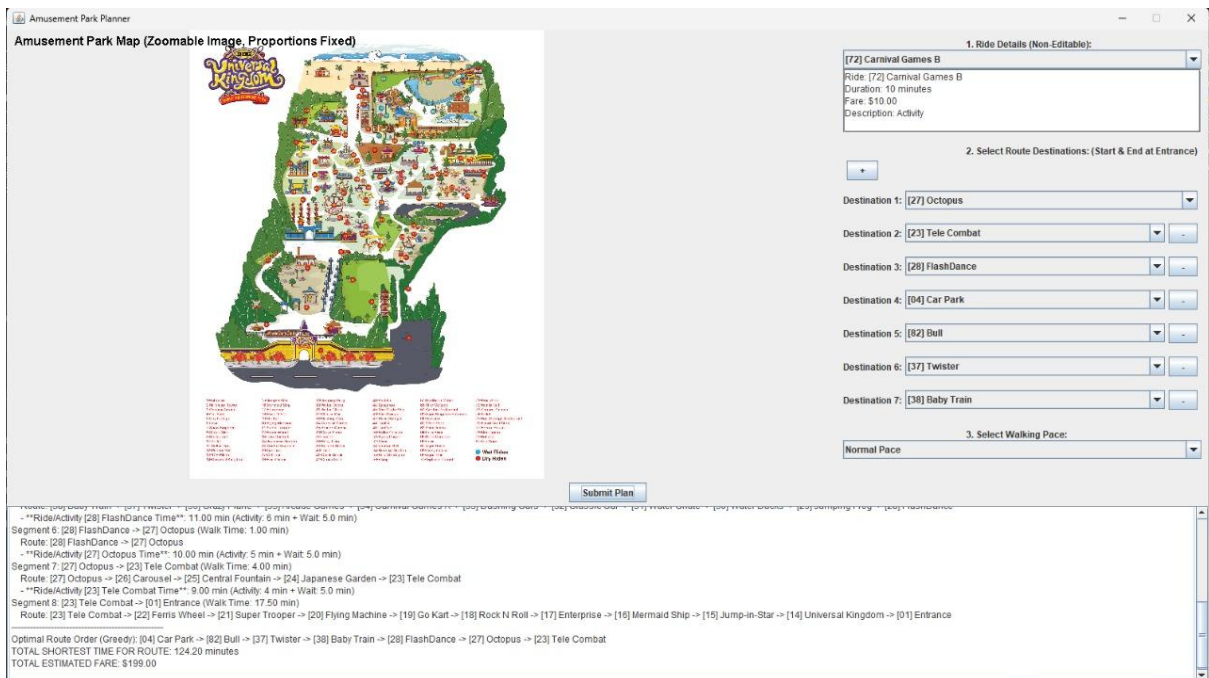


Fig 5.3



## CHAPTER 6

### CONCLUSION

The Theme Park Path Planner (TPPP) successfully evolved from a rudimentary maze solver into a sophisticated, customized routing system. By leveraging the **two-tiered approach of Dijkstra's Algorithm and the Greedy Nearest Neighbour Heuristic**, the project met its core objective of identifying a highly efficient path through a user-selected set of attractions. The system's ability to integrate user preferences (walking pace) with fixed attraction data (duration, fare) and execute rapid, segment-by-segment shortest path calculations provides a valuable, personalized itinerary planning tool. The successful implementation confirms the viability of applying combined graph search and heuristic optimization techniques to dynamic real-world environments, offering a tangible improvement in efficiency and user satisfaction for theme park visitors.

## REFERENCE

- <https://github.com/topics/maze-solver>
- <https://ojs.aaai.org/index.php/AIIDE/article/view/18733>
- [https://en.wikipedia.org/wiki/Maze-solving\\_algorithm](https://en.wikipedia.org/wiki/Maze-solving_algorithm)
- <https://ieeexplore.ieee.org/document/5152775>
- <https://ieeexplore.ieee.org/document/6194296>
- <https://ieeexplore.ieee.org/document/4082128>
- <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>
- <https://doi.org/10.1109/TCIAIG.2012.2197681>
- <https://doi.org/10.1109/TSSC.1968.300136>